

第15章

輪廓的檢測與匹配

15-1：影像內圖形的輪廓

- 15-1-1：找尋圖形輪廓findContours()
- 輪廓是指影像內圖形或是一些外形的邊緣線條。
- cv2.findContours()函數找出影像內部圖形輪廓時，需要將影像處理成黑與白的二值影像。所以也可以視為在黑影像中找尋白色物體。
- contours, hierarchy = cv2.findContours(image, mode, method)
 - contours: 在影像中找到的所有輪廓，資料類型是陣列(numpy.ndarray); 陣列中的元素則是輪廓內像素點座標
 - hierarchy: 輪廓間層次關係
 - image: 為8位元的單通道影像，如果原始影像是彩色必須轉為灰階影像。同時將灰階影像採用閾值處理，讓影像變成二值影像
 - mode: 輪廓的檢測模式(可參照下頁具名常數的設定表格)
 - method: 檢測輪廓的方法(可參照下頁具名常數的設定表格)

findContours() 找到的是「輪廓邊界的起始點」，不是看圖形中心點，也不是看圖形最左邊位置。

輪廓索引編號從0開始(contours), 但並不是從左到右偵測物件，而是依照 OpenCV 掃描影像時「發現輪廓的順序」

- **mode(輪廓的檢測模式):**

具名常數	值	說明
RETR_EXTERNAL	0	只檢測外部輪廓
RETR_LIST	1	檢測所有輪廓，但是不建立層次關係
RETR_CCOMP	2	檢測所有輪廓，同時建立兩個層級關係，如果內部還有輪廓則此輪廓與最外層的輪廓同級
RETR_TREE	3	檢測所有輪廓，同時建立一個樹狀層級關係

- **method:檢測輪廓的方法**

具名常數	值	說明
CHAIN_APPROX_NONE	1	儲存所有輪廓點
CHAIN_APPROX_SIMPLE	2	只有保存輪廓頂點的座標
CHAIN_APPROX_TC89L1	3	使用 Teh-Chin1 chain 近似的演算法
CHAIN_APPROX_TC89KCOS	4	使用 Teh-Chin1 chain 近似的演算法

PS.

Contours:

1. `cv2.CHAIN_APPROX_SIMPLE` 壓縮水平方向，垂直方向，對角線方向的元素，只保留該方向的終點座標，例如一個矩形輪廓只需4個點來儲存輪廓資訊。
2. 使用引數 `cv2.CHAIN_APPROX_NONE` 輪廓點有 1382 個，而使用引數 `cv2.CHAIN_APPROX_SIMPLE` 則輪廓點只有 4



15-1-2：繪製圖形的輪廓

- **findContours**是為了找到影像內圖形的輪廓
- **drawContours()**可以在影像內繪製圖形的輪廓
- **image** = cv2.drawContours(src_image, contours, contourIdx, color, thickness, lineType, hierarchy, maxLevel, offset)
 - **src_image** :
 - 執行前代表原始影像，執行drawContours後內容被同步更新，會與左邊**image**內的值相同。
 - 必須為8位元的單通道影像。
 - 如果是彩色影像需轉換成灰階影像，同時對灰階影採用閾值處理，讓影像變成二值影像。
 - **contours**:使用findContours所獲得的串列, 亦可視為需要繪製的輪廓
 - **contourIdx**: 列出需要繪製輪廓的索引。-1 表示繪製所有輪廓
 - **color**:使用BGR格式, 設定輪廓的顏色。
 - **thickness**: 繪製輪廓線條的粗細；如果為-1, 表示繪製實心
 - **lineType**: 繪製線條的類型
 - **hierarchy**: 可選擇參數, 這是findContours()輸出的層次關係
 - **maxLevel**: 繪製輪廓層次的深度, 如果是0表示繪製第0層
 - **offset**: 這是偏移參數, 可以設定所繪製輪廓的偏移量

15-2：繪製影像內圖形輪廓的系列實例

- 15-2-1：找尋與繪製影像內圖形輪廓的基本應用
- 1：使用`cv2.cvtColor( )`函數，將影像轉成灰階，讀者可以參考下列程式第7列。
- 2：使用`threshold( )`函數將灰階影像二值化，如果沒有特別考量，可以將中間值127當作閾值，讀者可以參考下列程式第9列。
- 程式實例：在影像`easy.jpg`內，輪廓檢測模式使用`RETR_EXTERNAL`，找尋圖形輪廓的應用，最後使用綠色繪製此輪廓。

15-2-2：認識findContours()函數的回傳值contours

- 利用findContours在影像中找到的所有輪廓，資料類型是串列(list)，串列內的元素則是輪廓內像素點座標
- 程式實例：使用easy.jpg列出所回傳contours的資料類型和長度。

```
import cv2
src = cv2.imread("easy.jpg")
src_gray = cv2.cvtColor(src,cv2.COLOR_BGR2GRAY) # 影像轉成灰階
ret, dst_binary = cv2.threshold(src_gray,127,255,cv2.THRESH_BINARY) # 二值化處理影像
# 找尋影像內的輪廓
contours, hierarchy = cv2.findContours(dst_binary,
                                     cv2.RETR_EXTERNAL,
                                     cv2.CHAIN_APPROX_SIMPLE)

print(f"資料類型      : {type(contours)}")
print(f"輪廓數量      : {len(contours)}")
```

```
===== RESTART: D:.
資料類型      : <class 'list'>
輪廓數量      : 3
```

```
import cv2
```

```
src = cv2.imread("easy.jpg")
```

```
cv2.imshow("src",src) # 顯示原始影像
```

```
src_gray = cv2.cvtColor(src,cv2.COLOR_BGR2GRAY) # 影像轉成灰階
```

```
# 二值化處理影像
```

```
ret, dst_binary = cv2.threshold(src_gray,127,255,cv2.THRESH_BINARY)
```

```
# 找尋影像內的輪廓
```

```
contours, hierarchy = cv2.findContours(dst_binary,  
                                     cv2.RETR_EXTERNAL,  
                                     cv2.CHAIN_APPROX_SIMPLE)
```

```
dst = cv2.drawContours(src,contours,-1,(0,255,0),5) # 繪製圖形輪廓, src執行前是原始影像,  
執行後是左邊的dst
```

```
cv2.imshow("result",dst)
```

```
# 顯示結果影像
```

```
cv2.imshow("src1",src)
```

```
# 再輸出一一次原始影像, src已非原圖而是與dst同更新
```

```
cv2.waitKey(0)
```

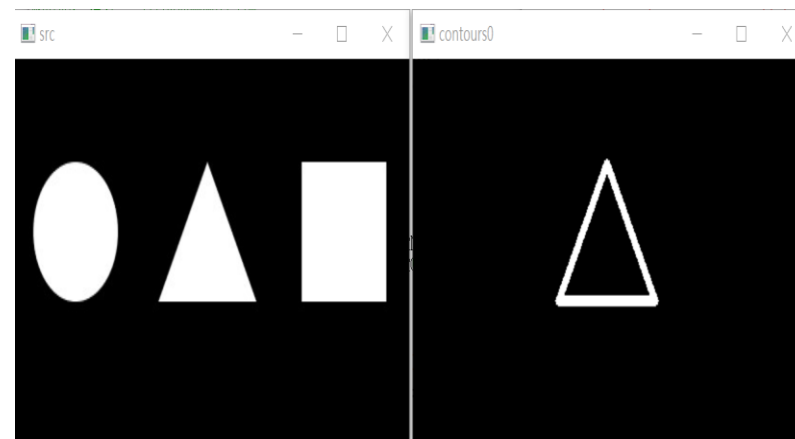
```
cv2.destroyAllWindows()
```

15-2-3：輪廓索引|contoursIdx

- 繪製輪廓使用`drawContours()`, 此函數第3個參數**contoursIdx**是輪廓的索引
- 但是分別繪製影像內的圖形輪廓。

■ `cv2.drawContours(image, contours, contourIdx, color, thickness)`

- Image: 要畫輪廓的圖像（原圖或空白圖）
- Contours: 輪廓資料，通常由 `cv2.findContours()` 取得
- contourIdx: 指定要畫第幾條輪廓（-1 表示畫出全部）
- Color: 輪廓線顏色，例如 (0, 255, 0) 為綠色輪廓線顏色，
- Thickness: 線條粗細（例如：2）若為 -1 則填滿輪廓內部



練習題

- 找圖

15-2-4：認識輪廓的屬性

- 使用`len()`函數計算輪廓的陣列長度；使用`shape`屬性,來獲得輪廓的外形
- 程式實例`ch15_4.py`：列出影像內圖片輪廓的屬性。
- `import cv2`
- `src = cv2.imread("easy.jpg")`
- `src_gray = cv2.cvtColor(src,cv2.COLOR_BGR2GRAY)` # 影像轉成灰階
- `ret, dst_binary = cv2.threshold(src_gray,127,255,cv2.THRESH_BINARY)` # 二值化處理影像
- `contours, hierarchy = cv2.findContours(dst_binary, cv2.RETR_EXTERNAL,`
- `cv2.CHAIN_APPROX_SIMPLE)` # 找尋影像內的輪廓
- `n = len(contours)` # 回傳輪廓數目
- `for i in range(n):` # 輸出輪廓的屬性
- `print(f'編號 = {i}')`
- `print(f'輪廓點的數量 = {len(contours[i])}')`
- `print(f'輪廓點的外形 = {contours[i].shape}')`

```
===== RESTART: D:/OpenCV_Python/ch15/ch15_4.py
編號 = 0
輪廓點的數量 = 146
輪廓點的外形 = (146, 1, 2)
編號 = 1
輪廓點的數量 = 4
輪廓點的外形 = (4, 1, 2)
編號 = 2
輪廓點的數量 = 134
輪廓點的外形 = (134, 1, 2)
```

- 實例5.py：列出編號1的輪廓點內容。

```
import cv2
```

```
src = cv2.imread("easy.jpg")
```

```
src_gray = cv2.cvtColor(src,cv2.COLOR_BGR2GRAY)    # 影像轉成灰階
```

```
# 二值化處理影像
```

```
ret, dst_binary = cv2.threshold(src_gray,127,255,cv2.THRESH_BINARY)
```

```
# 找尋影像內的輪廓
```

```
contours, hierarchy = cv2.findContours(dst_binary,  
                                       cv2.RETR_EXTERNAL,  
                                       cv2.CHAIN_APPROX_SIMPLE)
```

```
n = len(contours)
```

```
print(contours[1])
```

```
# 回傳輪廓數
```

```
===== RESTART: D:/OpenCV_Python/ch15/ch15_5.py
```

```
[[[300  65]]
```

```
[[[300 152]]
```

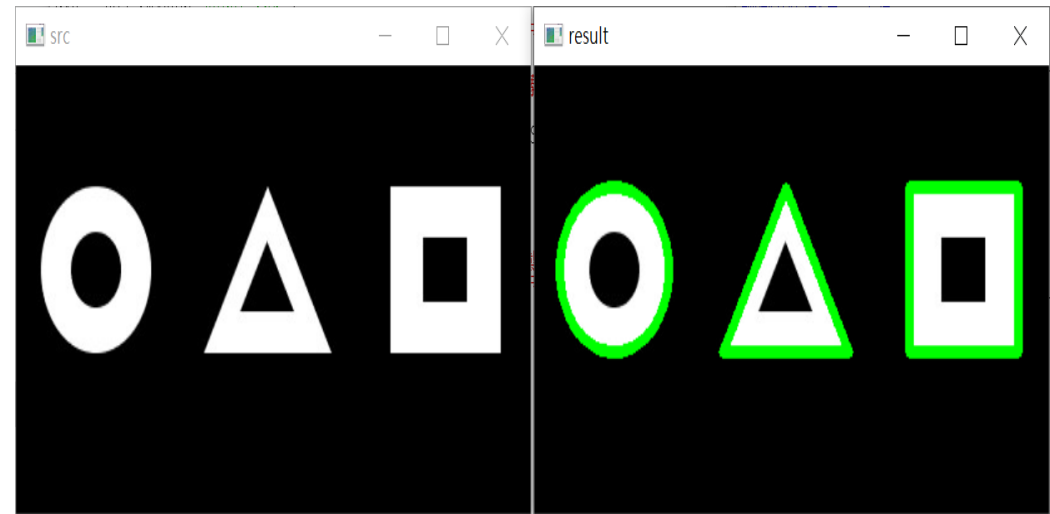
```
[[[387 152]]
```

```
[[[387  65]]]
```

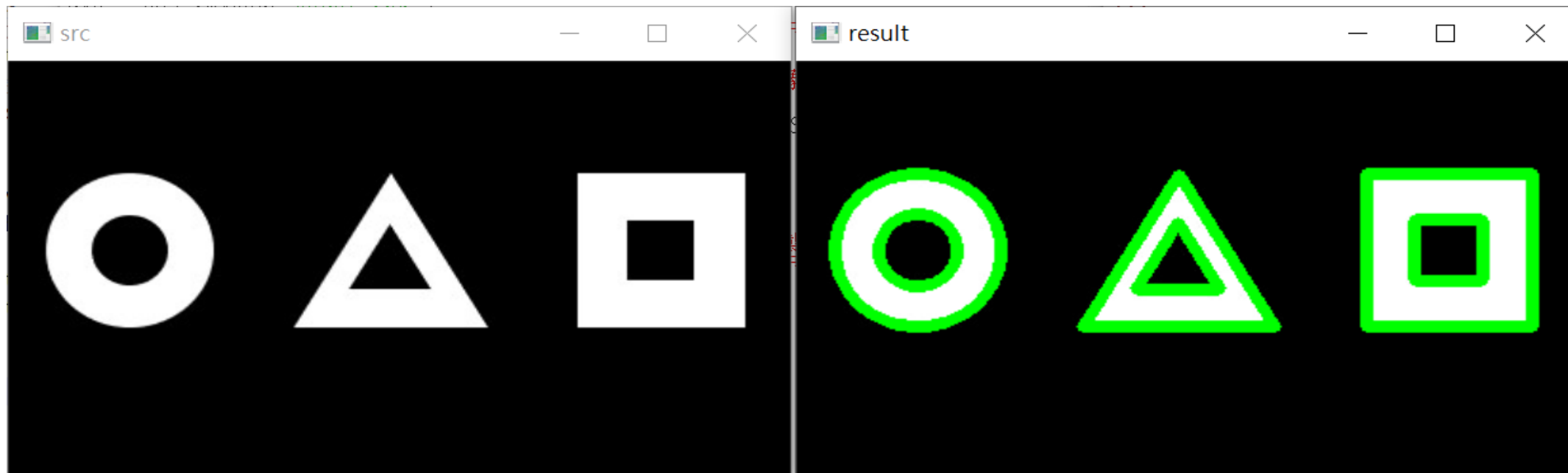
輪廓索引1, 外形是矩形,
外形由4個點代表輪廓

15-2-5：輪廓內有輪廓

- `import cv2`
- `src = cv2.imread("easy1.jpg")`
- `cv2.imshow("src",src)` # 顯示原始影像
- `src_gray = cv2.cvtColor(src,cv2.COLOR_BGR2GRAY)` # 影像轉成灰階
- # 二值化處理影像
- `ret, dst_binary = cv2.threshold(src_gray,127,255,cv2.THRESH_BINARY)`
- # 找尋影像內的輪廓
- `contours, hierarchy = cv2.findContours(dst_binary,`
- `cv2.RETR_EXTERNAL,`
- `cv2.CHAIN_APPROX_SIMPLE)`
- `dst = cv2.drawContours(src,contours,-1,(0,255,0),5)` # 繪製圖形輪廓
- `cv2.imshow("result",dst)` # 顯示結果影像
- `cv2.waitKey(0)`
- `cv2.destroyAllWindows()`



- 練習
- 請讀取easy1.jpg檔案, 檢測出所有圖形的輪廓。



- 練習題(找圖,秀輪廓):

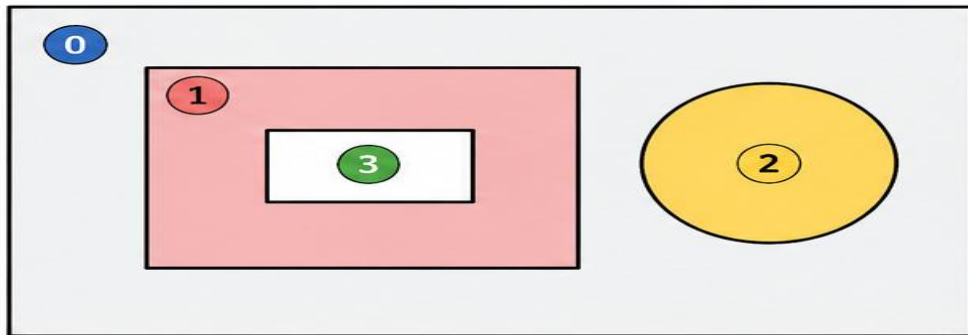
15-3：認識輪廓層級hierarchy

- `contours, hierarchy = cv2.findContours(image, mode, method)`
- 回傳的多維陣列資料型態 `hierarchy`，可以翻譯為層級。

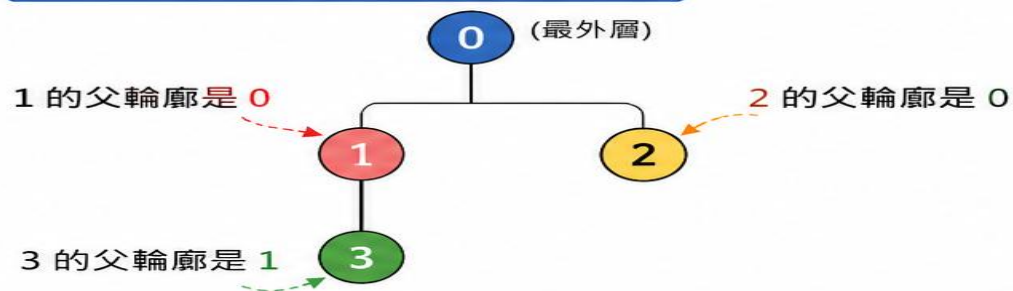
影像內圖形的輪廓：階層參數 (hierarchy) 如何解釋？

`hierarchy` 用來描述輪廓之間的「父子關係」與「同層關係」(階層結構)

概念圖



樹狀結構 (階層關係)



`hierarchy` 的含義 (每一列代表一個輪廓)

索引 i	Next 同層的 下一個輪廓	Prev 同層的 前一個輪廓	First_Child 第一個 子輪廓	Parent 父輪廓
0	-1	-1	1	-1
1	2	-1	3	0
2	-1	1	-1	0
3	-1	-1	-1	1

- **Next / Prev**：在同一層 (同一父輪廓下) 的兄弟輪廓之間的前後關係
- **First_Child**：該輪廓的第一個子輪廓 (沒有子輪廓為 -1)
- **Parent**：該輪廓的父輪廓 (最外層的父輪廓為 -1)

重點理解

- `hierarchy` 就像一棵樹，最外層輪廓是根 (Parent = -1)
- 子輪廓 (例如洞) 是某個輪廓的孩子 (First_Child 指向第一個孩子)
- 同層輪廓彼此是兄弟 (Next / Prev 連結)
- 有了 `hierarchy`，我們就能知道哪些輪廓是外框、哪些是內洞，並可控制是否保留洞或外框。

- `import cv2 # 15._10`
- `import numpy as np`
- `src = cv2.imread("lake.jpg")`
- `cv2.imshow("src",src)` # 顯示原始影像
- `src_gray = cv2.cvtColor(src,cv2.COLOR_BGR2GRAY)` # 影像轉成灰階
- # 二值化處理影像
- `ret, dst_binary = cv2.threshold(src_gray,150,255,cv2.THRESH_BINARY)`
- `cv2.imshow("binary",dst_binary)` # 顯示二值化影像
- # 找尋影像內的輪廓
- `contours, hierarchy = cv2.findContours(dst_binary,`
- `cv2.RETR_LIST,`
- `cv2.CHAIN_APPROX_SIMPLE)`
- `mask = np.zeros(src.shape, np.uint8)`
- `dst = cv2.drawContours(mask,contours,-1,(255,255,255),-1)` # 繪製圖形輪廓
- `dst_result = cv2.bitwise_and(src,mask)`
- `cv2.imshow("dst result",dst_result)` # 顯示結果影像 `cv2.waitKey(0)` `cv2.destroyAllWindows()`

15-3-1：檢測模式RETR_EXTERNAL

- 程式實例：使用easy2.jpg檔案，檢測模式採用RETR_EXTERNAL，列印hierarchy。

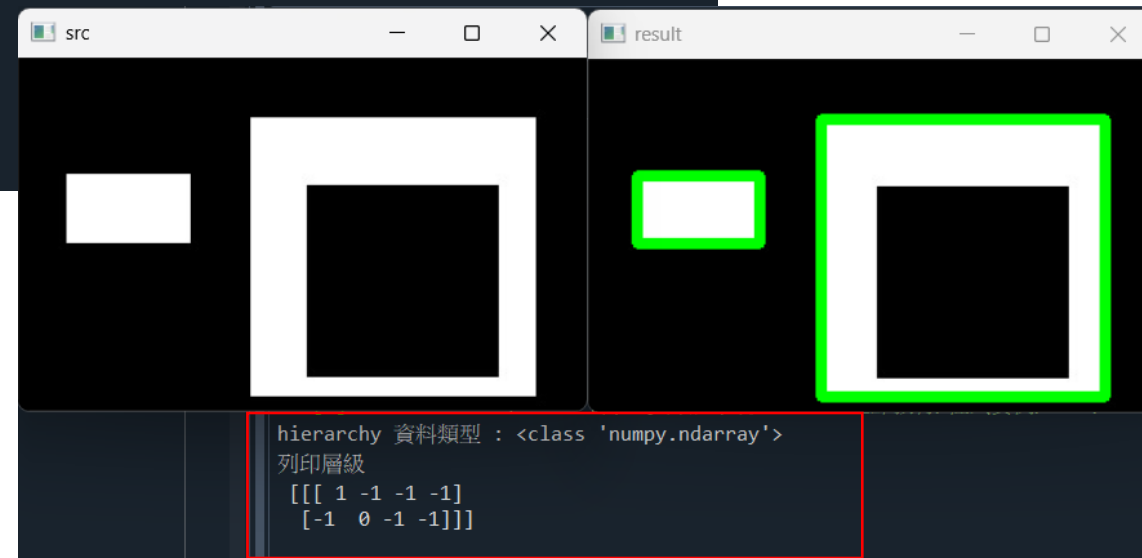
```
===== RESTART: D:/OpenCV_Python/ch15/ch15_11.py =====  
hierarchy 資料類型：<class 'numpy.ndarray'>  
列印層級  
[[[ 1 -1 -1 -1]  
  [-1  0 -1 -1]]]
```

```
import cv2

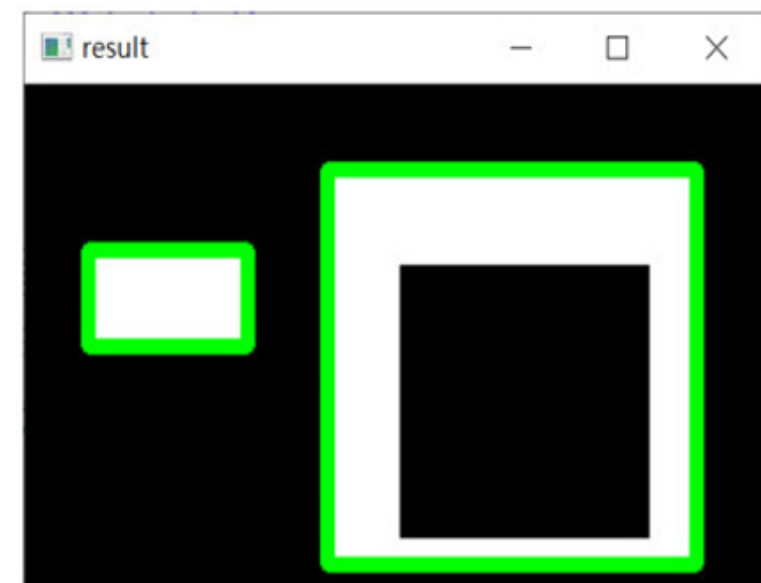
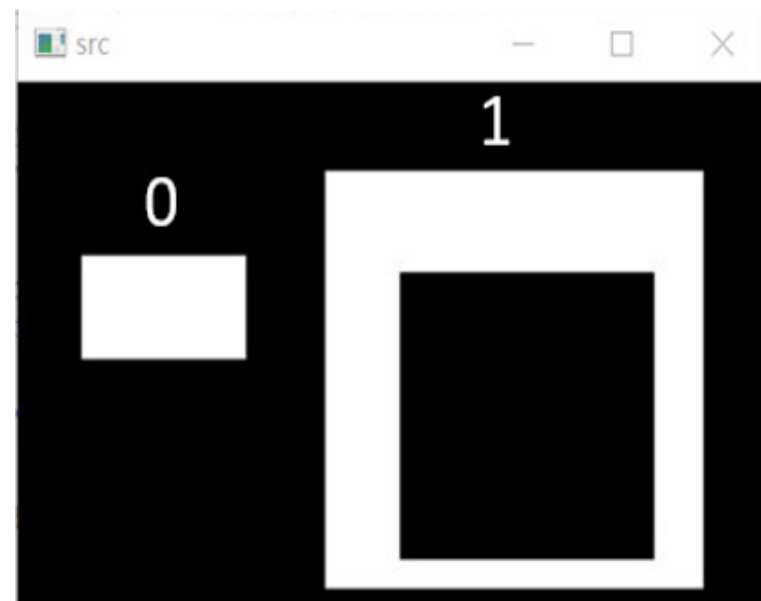
src = cv2.imread("easy2.jpg")
cv2.imshow("src",src) # 顯示原始影像

src_gray = cv2.cvtColor(src,cv2.COLOR_BGR2GRAY) # 影像轉成灰階
# 二值化處理影像
ret, dst_binary = cv2.threshold(src_gray,127,255,cv2.THRESH_BINARY)
# 找尋影像內的輪廓
contours, hierarchy = cv2.findContours(dst_binary,
                                     cv2.RETR_EXTERNAL,
                                     cv2.CHAIN_APPROX_SIMPLE)
dst = cv2.drawContours(src,contours,-1,(0,255,0),5) # 繪製圖形輪廓
cv2.imshow("result",dst) # 顯示結果影像
print(f"hierarchy 資料類型 : {type(hierarchy)}")
print(f"列印層級 \n {hierarchy}")

cv2.waitKey(0)
cv2.destroyAllWindows()
```



- hierarchy所回傳的陣列資料意義如下：
 - $[1 \ -1 \ -1 \ -1]$ ：這是第0個輪廓。
 - 1代表下一個同級輪廓的索引編號是1。
 - -1代表前一個同級輪廓不存在。
 - -1代表第一個子輪廓不存在。
 - -1代表父輪廓不存在。
- $[-1 \ 0 \ -1 \ -1]$ ：這是第1個輪廓。
- -1代表下一個同級輪廓不存在。
- 0代表前一個同級輪廓的索引編號是0。
- -1代表第一個子輪廓不存在。
- -1代表父輪廓不存在。



15-3-1：檢測模式RETR_LIST

- 程式實例ch15_12.py：使用easy2.jpg檔案重新設計ch15_11.py，檢測模式採用RETR_LIST，列印hierarchy。

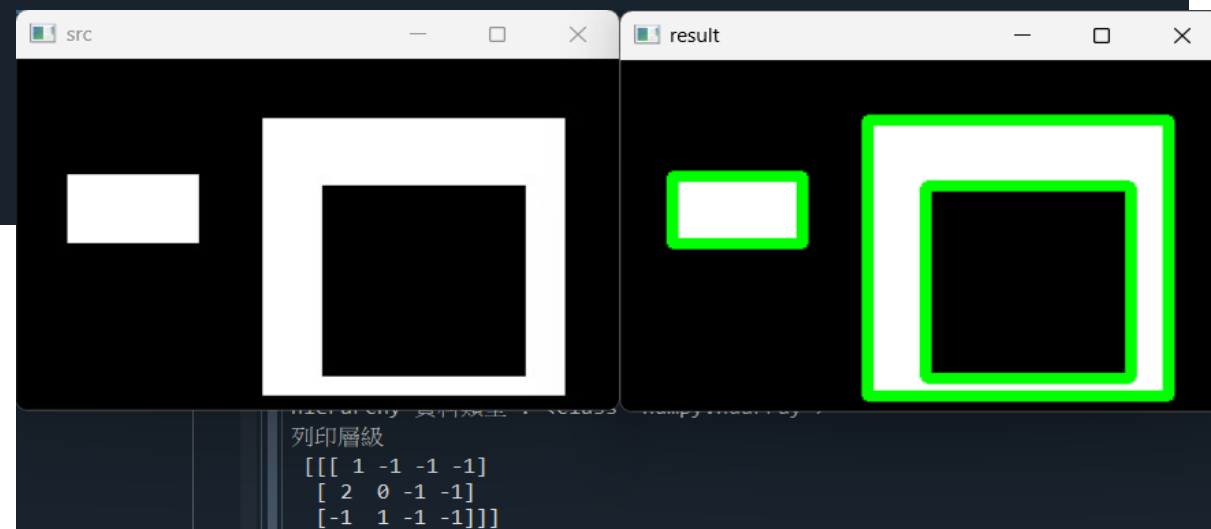
```
===== RESTART: D:/OpenCV_Python/ch15/ch15_12.py =====  
hierarchy 資料類型：<class 'numpy.ndarray'>  
列印層級  
[[[ 1 -1 -1 -1]  
 [ 2  0 -1 -1]  
 [-1  1 -1 -1]]]
```

```
import cv2

src = cv2.imread("easy2.jpg")
cv2.imshow("src",src) # 顯示原始影像

src_gray = cv2.cvtColor(src,cv2.COLOR_BGR2GRAY) # 影像轉成灰階
# 二值化處理影像
ret, dst_binary = cv2.threshold(src_gray,127,255,cv2.THRESH_BINARY)
# 找尋影像內的輪廓
contours, hierarchy = cv2.findContours(dst_binary,
                                     cv2.RETR_LIST,
                                     cv2.CHAIN_APPROX_SIMPLE)
dst = cv2.drawContours(src,contours,-1,(0,255,0),5) # 繪製圖形輪廓
cv2.imshow("result",dst) # 顯示結果影像
print(f"hierarchy 資料類型 : {type(hierarchy)}")
print(f"列印層級 \n {hierarchy}")

cv2.waitKey(0)
cv2.destroyAllWindows()
```



15-3-3：檢測模式RETR_CCOMP

- 當檢測模式採用RETR_CCOMP時，會檢測所有輪廓，同時建立兩個層級關係，下列將以實例解說。
- 程式實例ch15_13.py：使用easy3.jpg檔案，檢測模式採用RETR_CCOMP，列印輪廓與層次關係。

```
===== RESTART: D:\OpenCV_Python\ch15\ch15_13.py =====  
hierarchy 資料類型：<class 'numpy.ndarray'>  
列印層級  
[[[ 1 -1 -1 -1]  
  [-1  0  2 -1]  
  [ 3 -1 -1  1]  
  [-1  2 -1  1]]]
```

```
import cv2
```

```
src = cv2.imread("easy3.jpg")  
cv2.imshow("src",src)
```

顯示原始影像

```
src_gray = cv2.cvtColor(src,cv2.COLOR_BGR2GRAY)
```

影像轉成灰階

二值化處理影像

```
ret, dst_binary = cv2.threshold(src_gray,127,255,cv2.THRESH_BINARY)
```

找尋影像內的輪廓

```
contours, hierarchy = cv2.findContours(dst_binary,  
                                     cv2.RETR_CCOMP,  
                                     cv2.CHAIN_APPROX_SIMPLE)
```

```
dst = cv2.drawContours(src,contours,-1,(0,255,0),3) # 繪製圖形輪廓
```

顯示結果影像

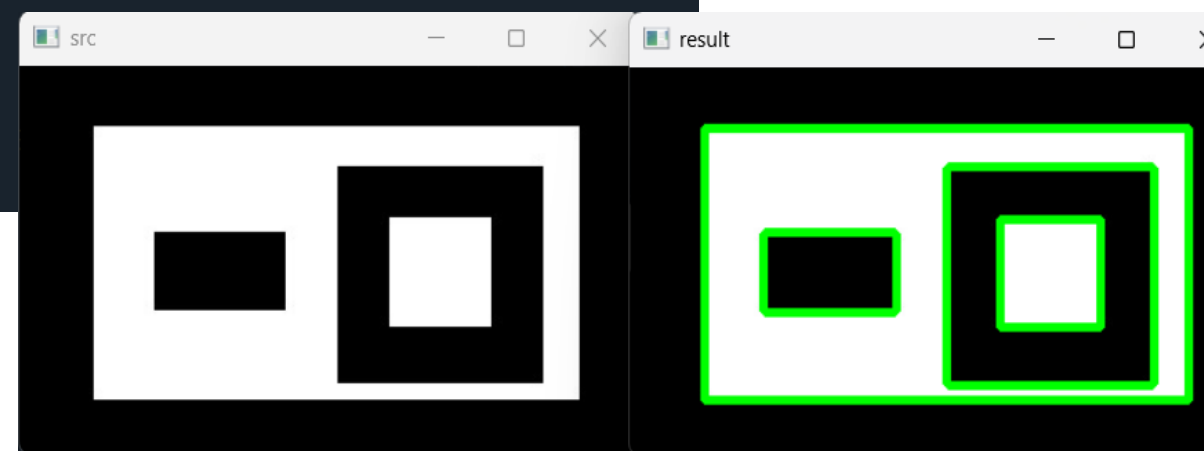
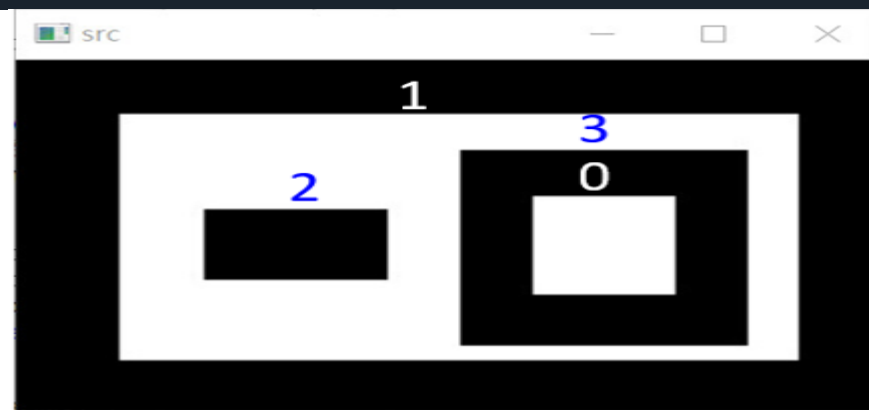
```
cv2.imshow("result",dst)
```

```
print(f"hierarchy 資料類型 : {type(hierarchy)}")
```

```
print(f"列印層級 \n {hierarchy}")
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```



列印層級

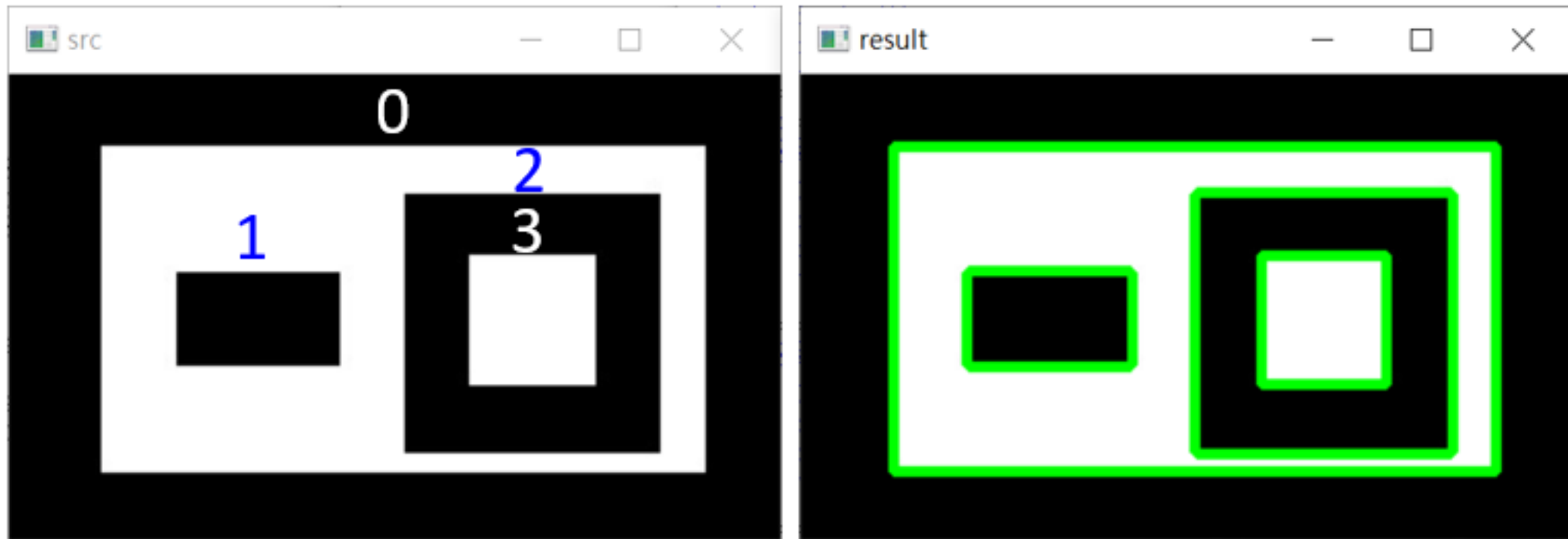
```
[[[ 1 -1 -1 -1]  
  [-1 0 2 -1]  
  [ 3 -1 -1 1]  
  [-1 2 -1 1]]]
```

- hierarchy所回傳的陣列資料意義如下：
- $[1 \ -1 \ -1 \ -1]$ ：這是第0個輪廓。
 - 1代表下一個同級輪廓的索引編號是1，因為這是內部增加的輪廓，此輪廓與最外部的輪廓同級。
 - -1代表前一個同級輪廓不存在。
 - -1代表第一個子輪廓不存在。
 - -1代表父輪廓不存在，因為在兩個層級觀念中，這相當於與最外圍輪廓同級。
- $[-1 \ 0 \ 2 \ -1]$ ：這是第1個輪廓。
 - -1代表下一個同級輪廓不存在。
 - 0代表前一個同級輪廓的索引編號是0。
 - 2代表第一個子輪廓的索引編號是2。
 - -1代表父輪廓不存在。

- $[3 -1 -1 1]$ ：這是第2個輪廓。
 - 3代表下一個同級輪廓的索引編號是3。
 - 1代表前一個同級輪廓不存在。
 - 1代表第一個子輪廓不存在。
 - 1代表父輪廓的索引編號是1。
- $[-1 2 -1 1]$ ：這是第3個輪廓。
 - 1代表下一個同級輪廓不存在。
 - 2代表前一個同級輪廓索引編號是2。
 - 1代表第一個子輪廓不存在，因為只有2層。
 - 1代表父輪廓的索引編號是1。

15-3-4：檢測模式RETR_TREE

- 採用RETR_TREE時，會檢測所有輪廓，同時建立樹狀層級關係



- $[-1 \ -1 \ 1 \ -1]$ ：這是第0個輪廓。
 - -1 代表下一個同級輪廓不存在。
 - -1 代表前一個同級輪廓不存在。
 - 1 代表第一個子輪廓的索引編號是1。
 - -1 代表父輪廓不存在。
- $[2 \ -1 \ -1 \ 0]$ ：這是第1個輪廓。
 - 2 代表下一個同級輪廓的索引編號是2。
 - -1 代表前一個同級輪廓不存在。
 - -1 代表第一個子輪廓不存在。
 - 0 代表父輪廓的索引編號是0。

- $[-1\ 1\ 3\ 0]$ ：這是第2個輪廓。
 - -1 代表下一個同級輪廓不存在。
 - 1 代表前一個同級輪廓的索引編號是 1 。
 - 3 代表第一個子輪廓的索引編號是 3 。
 - 0 代表父輪廓的索引編號是 0 。
- $[-1\ -1\ -1\ 2]$ ：這是第3個輪廓。
 - -1 代表下一個同級輪廓不存在。
 - -1 代表前一個同級輪廓不存在。
 - -1 代表第一個子輪廓不存在。
 - 2 代表父輪廓的索引編號是 2 。

15-4：輪廓的特徵－影像矩(Image moments)

- 輪廓的特徵是指質心、面積、周長、邊界框 ... 等，這些觀念常被使用在模式識別、影像識別
- Image moments 指輪廓的幾何特徵, 兩個輪廓是否相同最簡單的方式就是比較image moments (用數學方式描述一個圖形→數值特徵→Moments (矩))
 - $m = cv2.moments(array, binaryImage)$ → **$M = cv2.moments(contour)$**
 - Array:輪廓 contour,二值影像,點座標
 - binaryImage(是否把輸入視為二值影像): False

PS.contour 本身就是輪廓點集合→這是在計算輪廓 moments→binaryImage可以省略不寫

最常用於：物件辨識，形狀分析，質心計算，面積計算，形狀比對

- 輪廓面積計算： $M["m00"]$
- 質心 (Center) → 圖形中心點
$$cx = M["m10"] / M["m00"]$$
$$cy = M["m01"] / M["m00"]$$

二個方法的使用時機

`cv2.moments(contour)`

`cv2.moments(binary, True)`

使用時機

對某個輪廓計算 **moments**

對整張二值影像計算 **moments**

15-4-2：基礎影像矩推導－輪廓質心

- 簡單的說影像矩就是一組統計參數，這個參數紀錄像素所在位置與強度分佈

$$M_{ij} = \sum_x \sum_y x^i y^j I(x, y)$$

□ 面積計算

$$M_{00} = \sum_x \sum_y I(x, y)$$

□ 質心計算 (計算所有像素點的平均值)

$$(\bar{x}, \bar{y}) = \left(\frac{M_{10}}{M_{00}}, \frac{M_{01}}{M_{00}} \right)$$

M_{00} : 所有非0像素的總數

M_{10} : 所有非0 (x 座標) 的總和

M_{01} : 所有非0 (y 座標) 的總和

M_{01} : $1*1+2*1+3*1+4*1+1*1+2*1+3*1+4*1=20$

- 假設有一個4 x 4的二元值影像，假設內容與座標如下：

		y			
		1	2	3	4
x	1	0	0	0	0
	2	1	1	1	1
	3	1	1	1	1
	4	0	0	0	0

x	y	I(x,y)
2	1	1
2	2	1
2	3	1
2	4	1
3	1	1
3	2	1
3	3	1
3	4	1

1. 計算 M00：白色像素總數 → $M00 = 1+1+1+1+1+1+1+1 = 8$

→ 所以面積是：8

2. 計算 M10：x 座標加總

第 2 列有 4 個 1： $2+2+2+2 = 8$

第 3 列有 4 個 1： $3+3+3+3 = 12$ → $M10 = 8 + 12 = 20$

3. 計算 M01：y 座標加總

第 2 列的 y： $1+2+3+4 = 10$

第 3 列的 y： $1+2+3+4 = 10$

→ $M01 = 10 + 10 = 20$

$$M_{00} = \sum_x \sum_y I(x, y)$$

$$M_{10} = \sum_x \sum_y x I(x, y)$$

把每個白色像素的「**y**座標」加總

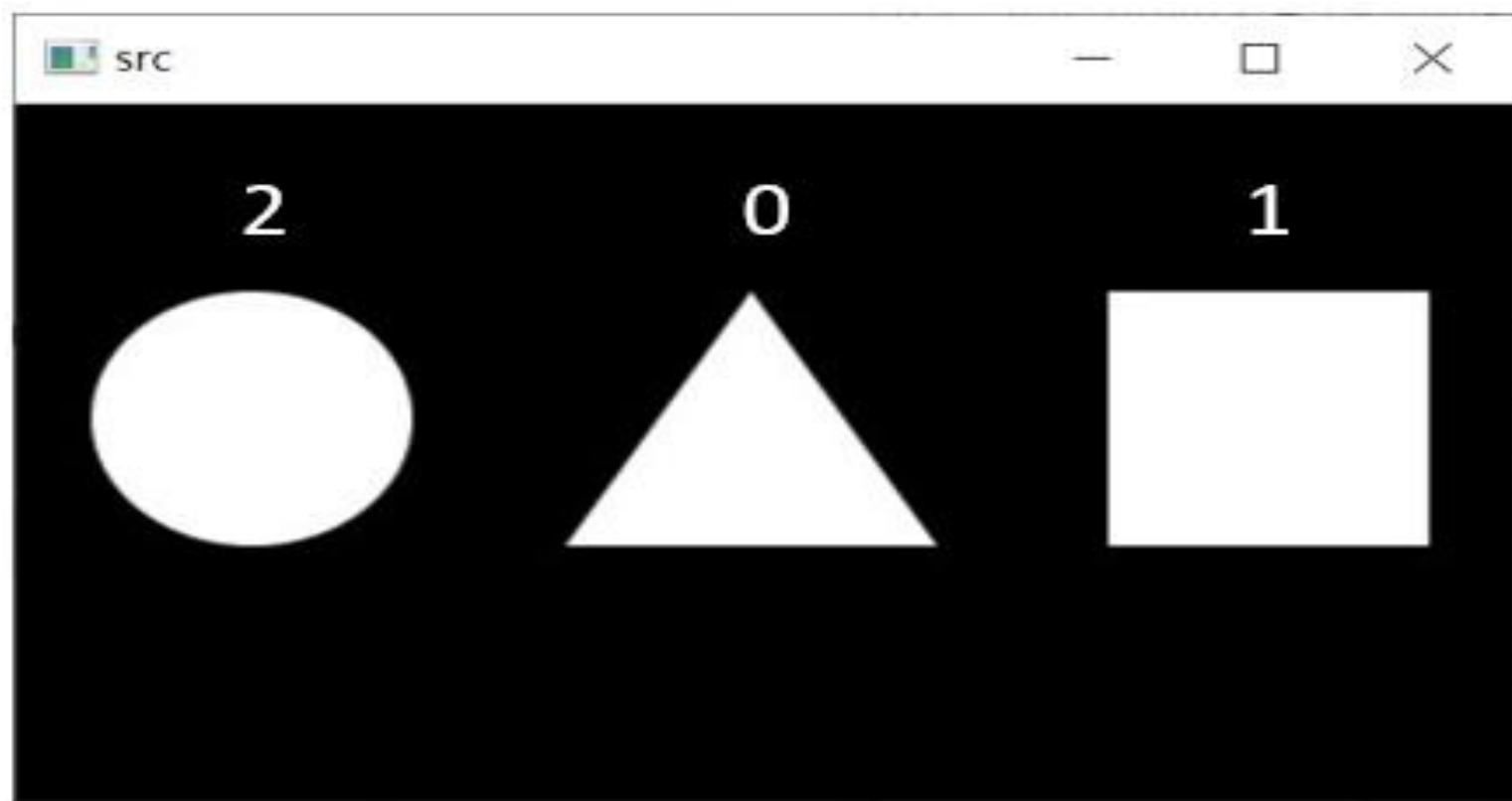
$$M_{01} = \sum_x \sum_y y I(x, y)$$

$$(\bar{x}, \bar{y}) = \left(\frac{M_{10}}{M_{00}}, \frac{M_{01}}{M_{00}} \right) = \left(\frac{20}{8}, \frac{20}{8} \right) = (2.5, 2.5)$$

15-4-4：計算輪廓面積

- `area = cv2.contourArea(contour, oriented)`
- 程式實例面積17.py：使用`contourArea()`函數計算輪廓面積。

```
===== RESTART: D:/OpenCV_Python/ch15/ch15_17.py =====  
輪廓 0 面積 = 4344.0  
輪廓 1 面積 = 7569.0  
輪廓 2 面積 = 5964.0
```



15-4-5：計算輪廓周長

- `area = cv2.arcLength(contour, closed)`
- 程式實例`ch15_18.py`：使用`arcLength()`函數計算輪廓周長。

```
===== RESTART: D:/OpenCV_Python/ch15/ch15_18.py =====  
輪廓 0 周長 = 315.4213538169861  
輪廓 1 周長 = 348.0  
輪廓 2 周長 = 289.42135322093964
```

```

import cv2

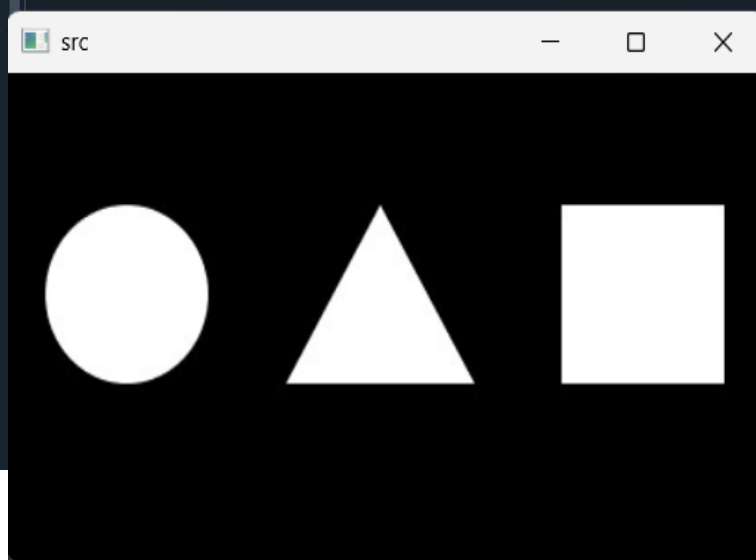
src = cv2.imread("easy.jpg")
cv2.imshow("src",src) # 顯示原始影像

src_gray = cv2.cvtColor(src,cv2.COLOR_BGR2GRAY) # 影像轉成灰階
# 二值化處理影像
ret, dst_binary = cv2.threshold(src_gray,127,255,cv2.THRESH_BINARY)
# 找尋影像內的輪廓
contours, hierarchy = cv2.findContours(dst_binary,
                                     cv2.RETR_EXTERNAL,
                                     cv2.CHAIN_APPROX_SIMPLE)

n = len(contours)
for i in range(n):
    M = cv2.moments(contours[i])
    area = cv2.arcLength(contours[i],True)
    print(f"輪廓 {i} 周長 = {area}")

cv2.waitKey(0)
cv2.destroyAllWindows()

```



```

輪廓 0 周長 = 315.4213538169861
輪廓 1 周長 = 348.0
輪廓 2 周長 = 289.42135322093964

```

15-5：輪廓外形的匹配 – Hu矩

- 15-5-1：OpenCV計算Hu矩的函數

- `hu = cv2.HuMoments(m)`

□ `hu`：這是Hu矩的回傳結果。

□ `m`：這是`moments( )`函數回傳的影像矩。

□ Hu矩是歸一化中心矩的線性組合，如果想要計算影像平移，縮放與旋轉保持不變的矩，就需要使用Hu矩, Hu具有平移,縮放,旋轉時保持不變的特性

$$h_0 = \eta_{20} + \eta_{02}$$

$$h_1 = (\eta_{20} - \eta_{02})^2 + 4\eta_{11}^2$$

$$h_2 = (\eta_{30} - 3\eta_{12})^2 + (3\eta_{21} - \eta_{03})^2$$

$$h_3 = (\eta_{30} + \eta_{12})^2 + (\eta_{21} + \eta_{03})^2$$

$$h_4 = (\eta_{30} - 3\eta_{12})(\eta_{30} + \eta_{12})[(\eta_{30} + \eta_{12})^2 - 3(\eta_{21} + \eta_{03})^2] \\ + (3\eta_{21} - \eta_{03})[3(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2]$$

$$h_5 = (\eta_{20} - \eta_{02})[(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2 + 4\eta_{11}(\eta_{30} + \eta_{12})(\eta_{21} + \eta_{03})]$$

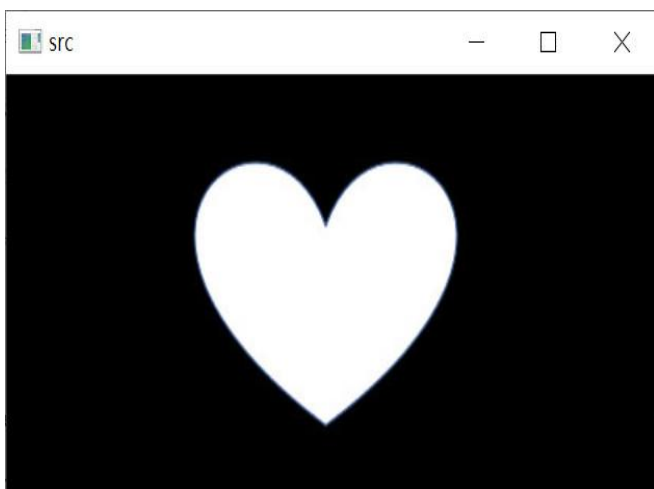
$$h_6 = (3\eta_{21} - \eta_{03})(\eta_{30} + \eta_{12})[(\eta_{30} + \eta_{12})^2 - 3(\eta_{21} + \eta_{03})^2] \\ + (\eta_{30} - 3\eta_{12})(\eta_{21} + \eta_{03})[3(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2]$$

15-5-2：第0個Hu矩的公式驗證

$$h_0 = \eta_{20} + \eta_{02}$$

$$h_0 - (\eta_{20} + \eta_{02}) = 0$$

- 程式實例ch15_19.py：使用heart.jpg，驗證上述公式。



```
歸一化中心矩 nu20 = 0.00047926293904721644  
歸一化中心矩 nu02 = 0.00023866024885601947  
Hu  
[[ 7.17923188e-04]  
 [ 5.78896885e-08]  
 [ 6.81194047e-11]  
 [ 2.42937256e-12]  
 [-3.12515353e-23]  
 [-5.84507960e-16]  
 [ 1.53613252e-25]]  
驗證結果 h0 - nu20 - nu02 = 0.0
```

```
import cv2  
  
src = cv2.imread("heart.jpg")  
cv2.imshow("src",src)                                     # 顯示原始影像  
  
src_gray = cv2.cvtColor(src,cv2.COLOR_BGR2GRAY)          # 影像轉成灰階  
M = cv2.moments(src_gray)                                  # 影像矩  
nu20 = M['nu20']  
print(f"歸一化中心矩 nu20 = {nu20}")  
nu02 = M['nu02']  
print(f"歸一化中心矩 nu02 = {nu02}")  
  
Hu = cv2.HuMoments(M)                                     # Hu矩  
print(f"Hu \n {Hu}")                                       # 列印Hu矩  
  
result = Hu[0][0] - (nu20 + nu02)                         # 驗證Hu矩 0, h0  
print(f"驗證結果 h0 - nu20 - nu02 = {result}")
```

- 程式實例ch15_20.py：有一幅影像3heart.jpg，請列出內部3個輪廓的Hu矩。在這3個輪廓中，使用findContours()函數所找的順序可以參考執行結果圖。

```
===== RESTART: D:\OpenCV_Python\ch15\ch15_20.py =====  
h0 = [0.18400164] [0.18362647] [0.18362647]  
h1 = [0.00390046] [0.00388969] [0.00388969]  
h2 = [0.00118707] [0.00115877] [0.00115877]  
h3 = [4.41012234e-05] [4.27240647e-05] [4.27240647e-05]  
h4 = [-1.00904583e-08] [-9.50599826e-09] [-9.50599826e-09]  
h5 = [-2.75427701e-06] [-2.66455047e-06] [-2.66455047e-06]  
h6 = [4.40391468e-11] [6.22527623e-11] [6.22527623e-11]
```



2

1

0

- 程式實例ch15_21.py：有一幅影像3shapes.jpg，請列出內部3個輪廓的Hu矩。在這3個輪廓中，使用findContours()函數所找的順序可以參考執行結果圖。



0

2

1

```
===== RESTART: D:/OpenCV_Python/ch15/ch15_21.py =====  
h0 = [0.18362125] [0.15949742] [0.18383141]  
h1 = [0.00388853] [6.2205636e-05] [0.00388999]  
h2 = [0.00115914] [1.09030421e-09] [0.00117116]  
h3 = [4.25428614e-05] [8.53260565e-13] [4.34904474e-05]  
h4 = [-9.44727464e-09] [1.09940012e-23] [-9.81486955e-09]  
h5 = [-2.6528849e-06] [1.82308256e-16] [-2.7123853e-06]  
h6 = [2.90666076e-11] [2.35891942e-23] [8.19693167e-11]
```

15-5-3：輪廓匹配

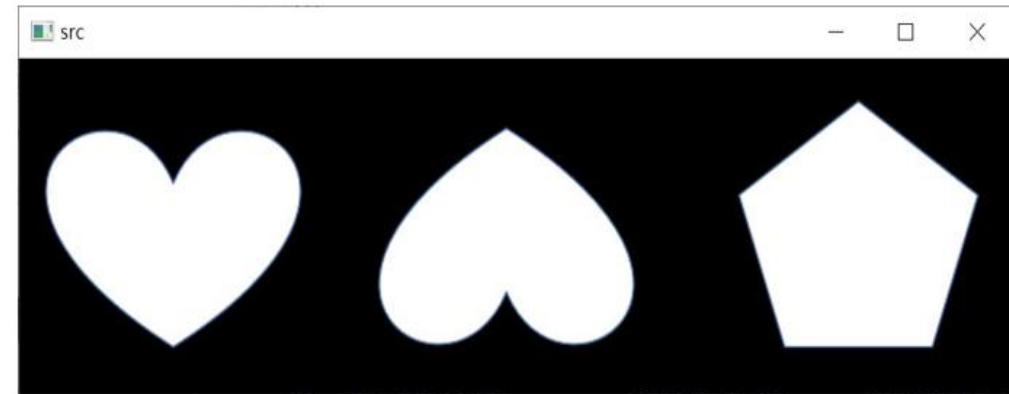
- `retval = cv2.matchShapes(contour1, contour2, method, parameter)`
- 使用 `myheart.jpg` 列出各影像的比較結果。

```
import cv2

src = cv2.imread("myheart.jpg")
cv2.imshow("src", src)
src_gray = cv2.cvtColor(src, cv2.COLOR_BGR2GRAY) # 影像轉成灰階
# 二值化處理影像
ret, dst_binary = cv2.threshold(src_gray, 127, 255, cv2.THRESH_BINARY)
# 找尋影像內的輪廓
contours, hierarchy = cv2.findContours(dst_binary,
                                       cv2.RETR_LIST,
                                       cv2.CHAIN_APPROX_SIMPLE)

match0 = cv2.matchShapes(contours[0], contours[0], 1, 0) # 輪廓0和0比較
print(f"輪廓0和0比較 = {match0}")
match1 = cv2.matchShapes(contours[0], contours[1], 1, 0) # 輪廓0和1比較
print(f"輪廓0和1比較 = {match1}")
match2 = cv2.matchShapes(contours[0], contours[2], 1, 0) # 輪廓0和2比較
print(f"輪廓0和2比較 = {match2}")

cv2.waitKey(0)
cv2.destroyAllWindows()
```



	0	1	2
輪廓0和0比較	=	0.0	
輪廓0和1比較	=	0.00046299603915214704	
輪廓0和2比較	=	0.29307592277155203	

15-6：輪廓外形匹配

- `sd = cv2.createShapeContextDistanceExtractor( )`
- `retval = sd.computeDistance(contour1, contour2)`
- 程式實例`ch15_23.py`：使用形狀場景距離重新設計`ch15_22.py`。

```

import cv2

# 讀取與建立影像 1
src1 = cv2.imread("mycloud1.jpg")
cv2.imshow("mycloud1",src1)
src1_gray = cv2.cvtColor(src1,cv2.COLOR_BGR2GRAY) # 影像轉成灰階
# 二值化處理影像
ret, dst_binary = cv2.threshold(src1_gray,127,255,cv2.THRESH_BINARY)
# 找尋影像內的輪廓
contours, hierarchy = cv2.findContours(dst_binary,
                                       cv2.RETR_LIST,
                                       cv2.CHAIN_APPROX_SIMPLE)

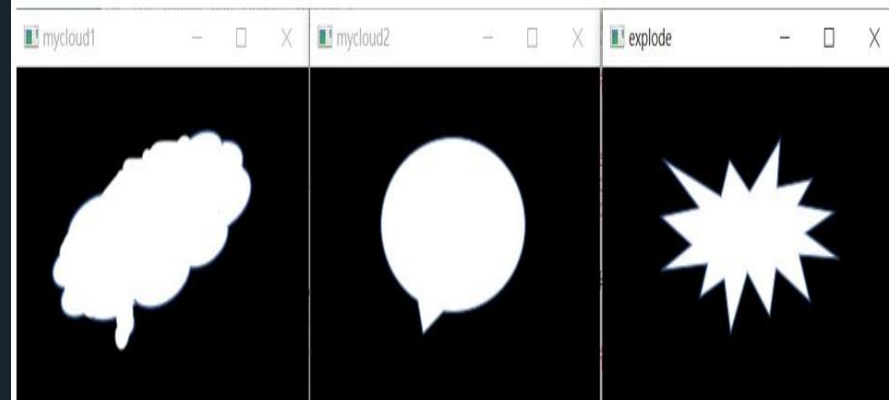
cnt1 = contours[0]
# 讀取與建立影像 2
src2 = cv2.imread("mycloud2.jpg")
cv2.imshow("mycloud2",src2)
src2_gray = cv2.cvtColor(src2,cv2.COLOR_BGR2GRAY) # 影像轉成灰階
ret, dst_binary = cv2.threshold(src2_gray,127,255,cv2.THRESH_BINARY)
contours, hierarchy = cv2.findContours(dst_binary,
                                       cv2.RETR_LIST,
                                       cv2.CHAIN_APPROX_SIMPLE)

cnt2 = contours[0]
# 讀取與建立影像 3
src3 = cv2.imread("explode1.jpg")
cv2.imshow("explode",src3)
src3_gray = cv2.cvtColor(src3,cv2.COLOR_BGR2GRAY) # 影像轉成灰階
ret, dst_binary = cv2.threshold(src3_gray,127,255,cv2.THRESH_BINARY)
contours, hierarchy = cv2.findContours(dst_binary,
                                       cv2.RETR_LIST,
                                       cv2.CHAIN_APPROX_SIMPLE)

cnt3 = contours[0]
sd = cv2.createShapeContextDistanceExtractor() # 建立形狀場景運算子
match0 = sd.computeDistance(cnt1, cnt1) # 影像1和1比較
print(f"影像1和1比較 = {match0}")
match1 = sd.computeDistance(cnt1, cnt2) # 影像1和2比較
print(f"影像1和2比較 = {match1}")
match2 =sd.computeDistance(cnt1, cnt3) # 影像1和3比較
print(f"影像1和3比較 = {match2}")
cv2.waitKey(0)
cv2.destroyAllWindows()

```

影像1和1比較 = 0.0
 影像1和2比較 = 0.22617380321025848
 影像1和3比較 = 0.8256418704986572



影像 1

影像 2

影像 3

15-6-2 : Hausdorff距離

$$h(A, B) = \max_{a \in A} \left\{ \min_{b \in B} \{d(a, b)\} \right\}$$

- `hd = cv2.createHausdorffDistanceExtractor( )`
- `retval = hd.computeDistance(contour1, contour2)`
- 程式實例 `ch15_24.py`：使用 Hausdorff 距離觀念重新設計 `ch15_23.py`。

```
===== RESTART: D:/OpenCV_Python/ch15/ch15_24.py =====  
影像1和1比較 = 0.0  
影像0和1比較 = 12.206555366516113  
影像0和2比較 = 8.5440034866333
```